

## Lecture 1: Frontier Overview (and Tokenization)

### Scratch the Frontier Problem

**Problem:** Researchers are becoming disconnected from the underlying technology.

- 2016: researchers implemented and trained their own models.
- 2018: researchers downloaded models (e.g., BERT) and fine-tuned them.
- 2026: researchers prompt API models (e.g., GPT/Claude/Gemini).

Moving up levels of abstraction boosts productivity, but

- these abstractions are leaky (in contrast to programming languages or operating systems).
- there is still frontier research to be done that requires tearing up the stack.

**Claim:** Full understanding of this technology is required for frontier research. But there's one small problem ... frontier models are very expensive.

- 2023: GPT-4 supposedly cost \$100M to train.
- 2025: xAI builds a cluster with 230K GPUs for training Grok.

There are no public details on how frontier models are built. We could build small language models (<1B parameters), but this might not be representative of large language models.

- Example 1: fraction of FLOPs spent in attention versus MLP changes with scale.
- Example 2: emergence of behavior with scale.

### Welcome to Language Modeling

**Question:** *What can we learn in this subject that transfers to frontier models?*

There are three types of knowledge:

- **Mechanics:** how things work (what a Transformer is, how model parallelism works).
- **Mindset:** squeezing the most out of the hardware, taking scaling seriously.
- **Intuitions:** which data and modeling decisions yield good accuracy.

We can teach mechanics and mindset (these do transfer). We can only partially teach intuitions (these do not necessarily transfer across scales).

**Remark:** *Some design decisions are simply not justifiable and just come from experimentation.*

For example, the Noam Shazeer paper that introduced SwiGLU [Shazeer 2020].

- Wrong interpretation: scale is all that matters, algorithms don't matter.
- Right interpretation: algorithms that scale are what matter.
- Mathematical abstraction: **accuracy** = **efficiency** × **resources**.

In fact, efficiency is way more important at larger scales (can't afford to be wasteful).

**Question:** *What is the best model one can build given a certain compute and data budget? In other words, maximize efficiency!*

## Current Research Landscape

### Pre-Neural (before 2010s)

- Language model to measure the entropy of English [Shannon 1950].
- N-gram language models (used in machine translation and speech recognition systems) [Brants+ 2007].

### Neural Ingredients (2010s)

- Long-Short Term Memory (LSTM) [Hochreiter+ 1997].
- First neural language model [Bengio+ 2003].
- Sequence-to-sequence modeling (for machine translation) [Sutskever+ 2014].
- Adam optimizer [Kingma+ 2014].
- Attention mechanism (for machine translation) [Bahdanau+ 2014].
- Transformer architecture (for machine translation) [Vaswani+ 2017].
- Mixture of experts [Shazeer+ 2017].
- Model parallelism [Huang+ 2018][Rajbhandari+ 2019][Shoeybi+ 2019].

### Early Foundation Models (late 2010s)

- ELMo: pretraining with LSTMs, fine-tuning improves downstream tasks [Peters+ 2018].
- BERT: pretraining with Transformers; fine-tuning improves downstream tasks [Devlin+ 2018].
- Google's T5 (11B): cast everything as text-to-text [Raffel+ 2019].

### Embracing Scaling

- OpenAI's GPT-2 (1.5B): fluent text, first signs of zero-shot [Radford+ 2019].
- Scaling laws: provide hope / predictability for scaling [Kaplan+ 2020].
- OpenAI's GPT-3 (175B): in-context learning [Brown+ 2020].
- Google's PaLM (540B): massive scale, undertrained [Chowdhery+ 2022].
- DeepMind's Chinchilla (70B): compute-optimal scaling laws [Hoffmann+ 2022].

### Open Models (2020s)

Early Attempts (attempts to replicate GPT-3):

- EleutherAI's open datasets (The Pile) and models (GPT-J) [Gao+ 2020][Wang+ 2021].
- Meta's OPT (175B): GPT-3 replication, lots of hardware issues [Zhang+ 2022].
- Hugging Face / BigScience's BLOOM (176B): focused on data sourcing [Workshop+ 2022].

Credible Open-Weight Models (weights + paper):

- Meta's Llama models [Touvron+ 2023][Grattafiori+ 2024].
- Mistral's models [Jiang+ 2023/2024].
- DeepSeek's models [DeepSeek-AI+ 2024/2025].
- Alibaba's Qwen models [Qwen+ 2024][Yang+ 2025].
- Moonshot's Kimi models [Kimi Team 2025/2026].
- Z.ai's GLM models [GLM Team 2025/2026].
- Minimax's models [MiniMax M2.5].
- Xiaomi's MIMO models [Xiaomi MIMO v2].

**Claim:** *These models are approaching closed models (GPT, Claude, Gemini, etc).*

Open-Source Models (weights + paper + code + data):

- AI2's Olmo models [Groeneveld+ 2024][Team Olmo 2024/2025].
- NVIDIA's Nemotron models [Parmar+ 2024][NVIDIA+ 2025].
- Marin's models (open development) [Marin 8B Retro][Marin 32B Retro].

Openness is important for trust and innovation. Ideas from open models enable us to teach CS336.

**Question:** *What is a language model?*

- 2018 (BERT): something you fine-tune.
- 2020 (GPT-3): something you prompt.
- 2022 (ChatGPT): something you talk to [example conversation].
- 2026 (agents): something that acts autonomously [example trace].

The fundamentals are the same (attention, kernels, optimization). The specifications are different (longer context, inference efficiency matters even more).

## Graduate Subject Logistics

**Why you should take this subject?**

- You have an obsessive need to understand how things work.
- You want to build up your research engineering muscles.

**Why you should not take this subject?**

- You actually want to get research done this quarter. (Talk to your advisor.)
- You are interested in learning about the hottest new techniques in AI (e.g., multimodality, RAG, etc). (You should take a seminar class for that.)
- You want to get good results in your own application domain. (You should just prompt or fine-tune an existing model.)

## Assignments

- 5 assignments (basics, systems, scaling laws, data, alignment).
- No scaffolding code, but we provide unit tests and adapter interfaces to help you check correctness.
- Implement locally to test for correctness, then run on cluster for benchmarking (accuracy and speed).
- Leaderboard for some assignments (minimize perplexity given training budget).

## Basics

### Tokenizer

What are the atoms that the model operates on? A tokenizer converts between raw inputs (bytes) and sequences of integers (tokens).

- Popular tokenizer: **Byte-Pair Encoding** (BPE) [Sennrich+ 2015].
- **Intuition:** break input into frequently-occurring chunks.
- Efficiency lens: Reduce context length (1000 bytes  $\rightarrow$  250 tokens).
- Adaptive computation (more modeling capacity on interesting parts of input).

**Dream:** We want tokenizer-free model architectures, which operate directly on bytes. These are promising, but have not yet been scaled up to the frontier.

### Model architecture

How does the model "think" with respect to machine learning and neural networks?

- Starting point: original Transformer.
- Activation functions: ReLU, SwiGLU.
- Positional encodings: sinusoidal, RoPE.
- Normalization: LayerNorm, RMSNorm, QK norm, pre-norm versus post-norm.
- Attention: full, sparse/local, group-query (GQA), multi-head latent (MLA) attention.
- Recurrence/state-space models/linear attention: Mamba, Gated DeltaNet.
- MLP: dense, mixture of experts.
- Shape (hidden dimension, depth, number of heads, number of experts).

## Training

How do you set the parameters of the model?

- Loss function (e.g., multi-token prediction).
- Optimizer (e.g., AdamW, SOAP, Muon).
- Initialization scale (e.g., Xavier init, muP).
- Learning rate schedule (e.g., cosine, WSD).
- Regularization (e.g., dropout, weight decay).
- Batch size (e.g., critical batch size).
- MoE specific: load balancing (e.g., aux-free).

## Assignment 1 (basics)

- Implement BPE tokenizer.
- Implement Transformer, cross-entropy loss, AdamW optimizer, training loop.
- Do resource accounting.
- Train on TinyStories and OpenWebText.
- Leaderboard: minimize OpenWebText perplexity given 45 minutes on a B200.

Everything is about balancing the following:

- expressivity (can represent complex dependencies in the data).
- stability (keep parameter and gradient norms in goldilocks zone).
- efficiency (runs fast on hardware, both training and inference).

## Systems

### Hardware

- Resource accounting: compute and memory characteristics of a model.
- Model parameters must be moved from memory (HBM) to the compute (SMs).
- Roofline analysis: understand whether we're compute-bound or memory-bound.
- Benchmarking and profiling (nsight): see what happens in practice.

## Kernels

- Kernel is a function that runs on GPU.
- When using PyTorch, each primitive operation launches a standard kernel.
- Can write custom kernels to make GPUs go brrr.
- Principle: organize computation to minimize data movement.
- Naive: read HBM; compute A; write HBM; read HBM; compute B; write HBM.
- Fused: read HBM; compute A and B; write HBM.
- Strategies: operator fusion (matmul + activation), tiling (FlashAttention).
- Warp divergence, memory coalescing, bank conflicts, occupancy, bulk-async memory.
- Write kernels in CUDA/Triton/CUTLASS/ThunderKittens.

## Parallelism

- What if we have 1024 GPUs?
- Data movement between GPUs is even slower, but same 'minimize data movement' principle holds.
- Use classic collective operations (e.g., gather, reduce, all-reduce).
- Shard memory (parameters, activations, gradients, optimizer states) across GPUs.
- How to split computation: {data, tensor, pipeline, sequence, expert} parallelism.

## Inference

Generate tokens given a prompt (needed to actually use models!):

- Inference is also needed for reinforcement learning, test-time compute, evaluation.
- Two phases: prefill and decode.
- Prefill (similar to training): tokens are given, can process all at once (compute-bound).
- Decode: need to generate one token at a time (memory-bound).

Methods to speed up decoding:

- Use cheaper model (via model pruning, quantization, distillation).
- Speculative decoding: use a cheaper "draft" model to generate multiple tokens, then use the full model to score in parallel (exact decoding!).
- Systems optimizations: fused kernels, continuous batching.

**Assignment 2 (systems)**

- Implement a fused RMSNorm kernel in Triton.
- Implement distributed data parallel training.
- Implement optimizer state sharding.
- Benchmark and profile the implementations.

Recommended book: **How to Scale Your Model!**

- Nicely lays out how to approach systems for LLMs conceptually.
- From Google, so it foregrounds TPUs, but high-level concepts are similar.

**Scaling**

If you had  $1e25$  FLOPs of compute, what hyperparameters would you use to train a good model?  
Too expensive to do hyperparameter tuning at full scale!

**Key Concept:** Instead of a single scale, think of a scaling recipe (FLOPs  $\rightarrow$  hyperparameters).

- Run experiments to compute the loss at various smaller scales (e.g., up to  $1e24$  FLOPs).
- Fit a scaling law to predict the loss of the scaling recipe at the target scale (e.g.,  $1e25$  FLOPs).

Now you can:

1. Optimize the scaling recipe targeting a larger scale using smaller scale experiments.
2. Predict the loss at the target scale before actually running the experiment!

Scaling laws don't happen automatically, they require careful construction of a scaling recipe. Parameterize the model in a way to get hyperparameter transfer [Yang+ 2022]. Predictability is at least as important as optimality!

**Question:** Given a FLOPs budget ( $C = 6 N D$ ), use a bigger model ( $N$ ) or train on more tokens ( $D$ )?

Classic compute-optimal scaling laws:

- ISOFLOP curves: for multiple small FLOPs budgets, find optimal  $N$ .
- Then fit a scaling law to extrapolate to large FLOPs budgets.

TL;DR:  $D = 20 N$  is roughly optimal (e.g., 70B parameter model should be trained on 1.4T tokens).

**Remark:** *This doesn't take into account inference costs (want a smaller model).*

### Assignment 3 (scaling)

- We define a training API (hyperparameters  $\rightarrow$  loss) based on previous runs.
- Submit "training jobs" (under a FLOPs budget) and gather data points.
- Fit scaling laws to the data points.
- Submit extrapolated hyperparameters and loss predictions.
- Leaderboard: minimize loss given FLOPs budget.

### Data

What capabilities do we want the model to have? Multilingual? Good at conversation? Agentic coding capabilities?

### Evaluation

What is the purpose of evaluation?

1. Internal: guide development (smoothness across scales, relative performance matters).
2. External: measure absolute quality of a real use case (ecological validity matters).

Examples of evaluations:

1. Perplexity: ideally run on private documents not on Internet (avoid contamination).
2. Advanced use cases: GPQA, HLE, SWE-Bench, Terminal-Bench.

LMs are general purpose, require a diverse set of evaluations!

### Data curation

- Data does not just fall from the sky.
- Sources: webpages crawled from the Internet, books, arXiv papers, GitHub code, etc.
- Appeal to fair use to train on copyright data?
- Might have to license data (e.g., Google with Reddit data).
- Raw data is HTML, PDF, directories (not text), requires processing.

**Data processing**

- Transformation: convert HTML / PDF to text (extract main content).
- Filtering: keep high quality data, remove harmful content (via classifiers).
- Deduplication: save compute, avoid memorization; use Bloom filters or MinHash.
- Data mixing: how much to upweight / downweight each source?
- Rewriting / synthetic data: use LM to augment real data, more similar to downstream tasks.

Types of data:

- Pretraining data: large and diverse.
- Mid-training data: high quality, including long-context.
- Post-training data: supervised fine-tuning (conversations, agentic traces with tool calling).

**Assignment 4 (data)**

- Convert Common Crawl HTML to text.
- Train classifiers to filter for quality and harmful content.
- Deduplication using MinHash.
- Leaderboard: minimize perplexity given token budget.

**Alignment**

So far, we have trained a model on full supervision (predict the next token). Now that the model should be reasonable, we can improve it further from weak supervision. Why weak supervision? When it is easier to critique than to generate.

Template:

1. Generate responses from the model.
2. Score responses with a human, verifier, LM judge.
3. Update the model to prefer better responses.

Algorithms:

- Proximal Policy Optimization (PPO) from reinforcement learning.

- Direct Policy Optimization (DPO): for preference data, simpler.
- Group Relative Preference Optimization (GRPO): remove value function.

Challenges:

- RL algorithms are unstable and hard to tune.
- At scale, this requires a lot of new infrastructure (inference with async rollouts).
- Constantly trading off systems efficiency and on-policy-ness.

### Assignment 5 (alignment)

- Implement Direct Preference Optimization (DPO).
- Implement Group Relative Preference Optimization (GRPO).

Remember it's all about **efficiency**:

- Resources: data + hardware (compute, memory, communication bandwidth).
- How do you train the best model given a fixed set of resources?

Today, we are **compute-constrained**, so design decisions will reflect squeezing the most out of given hardware.

- Systems: clearly about efficiency.
- Tokenization: working with raw bytes is elegant, but compute-inefficient with today's model architectures.
- Model architecture: many changes motivated by reducing memory or FLOPs (e.g., sharing KV caches, sliding window attention).
- Data filtering: avoid wasting precious compute updating on bad / irrelevant data.
- Scaling laws: use less compute on smaller models to do hyperparameter tuning.

Tomorrow, we will become **data-constrained**...

## Tokenization

This unit was inspired by Andrej Karpathy's video on tokenization; check it out!

## Introduction to Tokenization

Raw text is generally represented as Unicode strings.

```
string = "Hello, World!"
```

A language model places a probability distribution over sequences of tokens (usually represented by integer indices).

```
indices = [15496, 11, 995, 0]
```

So we need a procedure that *encodes* strings into tokens and a procedure that *decodes* tokens back into strings. A **Tokenizer** is a class that implements the encode and decode methods.

```
class Tokenizer(ABC):  
    """Abstract interface for a tokenizer."""  
    def encode(self, string: str) -> list[int]:  
        raise NotImplementedError  
  
    def decode(self, indices: list[int]) -> str:  
        raise NotImplementedError
```

## Tokenization Examples

To get a feel for how tokenizers work, play with this [\[interactive site\]](#).

- A word and its preceding space are part of the same token (e.g., " world").
- A word at the beginning and in the middle are represented differently (e.g., "hello hello").
- Numbers are tokenized into every few digits.

Here's the GPT-5 tokenizer from OpenAI (tiktoken) in action.

```
tokenizer = tiktoken.get_encoding("o200k_base")  
string = "Hello, World!"
```

Check that encode() and decode() roundtrip:

```
indices = tokenizer.encode(string)  
reconstructed_string = tokenizer.decode(indices)  
assert string == reconstructed_string
```

Compression ratio: number of bytes per token.

```
def get_compression_ratio(string: str, indices: list[int]) -> float:
    """Input: `string` that has been tokenized into `indices`.
       Output: the number of UTF-8 bytes per token."""
    num_bytes = len(bytes(string, encoding="utf-8"))
    num_tokens = len(indices)
    return num_bytes / num_tokens
```

**Remark:** *The larger the compression ratio, the shorter the sequence (good since attention is quadratic in sequence length).*

One could increase compression ratio by increasing vocabulary size (number of possible token values increases), leading to sparsity.

```
vocabulary_size = tokenizer.n_vocab
```

### Character Tokenizer

A (Unicode) string is a sequence of (Unicode) characters. Each character can be converted into a code point (integer) via **ord**.

```
assert ord("A") == 65
assert ord("a") == 97
```

It can be converted back via **chr**.

```
assert chr(65) == "A"
assert chr(97) == "a"
```

Now let's build a **Tokenizer** and make sure it round-trips:

```
class CharacterTokenizer(Tokenizer):
    """Represent a string as a sequence of Unicode code points."""
    def encode(self, string: str) -> list[int]:
        return list(map(ord, string))

    def decode(self, indices: list[int]) -> str:
        return "".join(map(chr, indices))
```

There are approximately 150K Unicode characters.

```
vocabulary_size = max(indices) + 1 # This is a lower bound
```

Problems:

1. This is a very large vocabulary.
2. Many characters are quite rare, which is inefficient use of the vocabulary.

This tokenizer is the worst of both worlds (large vocabulary, low compression ratio).

## Byte Tokenizer

Unicode strings can be represented as a sequence of bytes, which can be represented by integers between 0 and 255. The most common Unicode encoding is [UTF-8].

Some Unicode characters are represented by one byte:

```
assert bytes("a", encoding="utf-8") == b"a"
```

Others take multiple bytes:

```
assert bytes("CS336", encoding="utf-8") == b"\xf0\x9f\x8c\x8d"
```

Now let's build a **Tokenizer** and make sure it round-trips:

```
class ByteTokenizer(Tokenizer):
    """Represent a string as a sequence of bytes."""
    def encode(self, string: str) -> list[int]:
        string_bytes = string.encode("utf-8")
        indices = list(map(int, string_bytes))
        return indices

    def decode(self, indices: list[int]) -> str:
        string_bytes = bytes(indices)
        string = string_bytes.decode("utf-8")
        return string
```

The vocabulary is nice and small: a byte can represent 256 values.

```
vocabulary_size = 256
```

What about the compression rate?

```
assert compression_ratio == 1
```

The compression ratio is terrible, which means the sequences will be too long. Given that the context length of a Transformer is limited (since attention is quadratic), this is not looking great ...

## Word Tokenizer

Another approach (closer to what was done classically in NLP) is to split strings into words.

```
string = "I'll say supercalifragilisticexpialidocious!"
```

This regular expression keeps all alphanumeric characters together (words).

```
chunks = regex.findall(r"\w+|.", string)
```

To turn this into a **Tokenizer**, we need to map these chunks into integers.

What's good: each token is meaningful (since humans invented words).

```
vocabulary_size = "Number of distinct chunks in training data"
```

Compression ratio is good, but vocabulary size can be huge.

- Many words are rare and the model won't learn much about them.
- This doesn't obviously provide a fixed vocabulary size.
- New words we haven't seen during training get a special UNK token, which is ugly and can mess up perplexity calculations.

## Byte-Pair Tokenizer

The BPE algorithm was introduced by Philip Gage in 1994 for data compression.

- It was adapted to NLP for neural machine translation.
- Previously, papers had been using word-based tokenization.
- BPE was then used by GPT-2.

Lemma: train the tokenizer on raw text to construct a vocabulary tailored to the data.

```
class BPETokenizer(Tokenizer):
    """BPE tokenizer given a set of merges and a vocabulary."""
    def __init__(self, params: BPETokenizerParams):
        self.params = params

    def encode(self, string: str) -> list[int]:
        indices = list(map(int, string.encode("utf-8")))
        # Note: this is a very slow implementation
        for pair, new_index in self.params.merges.items():
            indices = merge(indices, pair, new_index)
        return indices

    def decode(self, indices: list[int]) -> str:
        bytes_list = list(map(self.params.vocab.get, indices))
        string = b"".join(bytes_list).decode("utf-8")
        return string
```

Proof: start with each byte as a token, and successively merge the most common pair of adjacent tokens.

```
def train_bpe(string: str, num_merges: int) -> BPETokenizerParams:
    """Start with the list of bytes of `string`."""
    indices = list(map(int, string.encode("utf-8")))
    merges: dict[tuple[int, int], int] = {}
    vocab: dict[int, bytes] = {x: bytes([x]) for x in range(256)}
    for i in range(num_merges):
        # Count the number of occurrences of each pair of tokens
        counts = count_adjacent_pairs(indices)
        # Find the most common pair
        pair = max(counts, key=counts.get)
        # Merge that pair
        new_index = 256 + i
        merges[pair] = new_index
        vocab[new_index] = vocab[pair[0]] + vocab[pair[1]]
        indices = merge(indices, pair, new_index)
    compression_ratio = get_compression_ratio(string, indices)
    return BPETokenizerParams(vocab=vocab, merges=merges)

def count_adjacent_pairs(indices: list[int]) -> BPETokenizerMerges:
    """Return a dictionary mapping each adjacent pair of tokens in
    `indices` to the number of times it occurs."""
    counts = defaultdict(int)
    for index1, index2 in zip(indices, indices[1:]):
        counts[(index1, index2)] += 1
    return counts

def merge(indices: list[int],
          pair: tuple[int, int], new_index: int) -> list[int]:
    """Return `indices`, but all instances of `pair` replaced with
    `new_index`."""
    new_indices, i = [], 0
    while i < len(indices):
        if (i + 1 < len(indices) and indices[i] == pair[0]
            and indices[i + 1] == pair[1]):
            new_indices.append(new_index)
            i += 2
        else:
            new_indices.append(indices[i])
            i += 1
    return new_indices
```